

Task 3: Train-Test Split & Data Preprocessing

Dataset Used: Iris Dataset

The Iris dataset is used for this task to prepare data for Machine Learning using the **Scikit-learn** library. The dataset contains 150 flower records with measurements of sepal and petal dimensions and the corresponding flower species.

The main objective of this task is to divide the dataset into training and testing sets and apply basic preprocessing so that the data is ready for Machine Learning.

1. Import Required Libraries

The required Python libraries were imported using Scikit-learn and Pandas. Pandas is used for handling the dataset, while Scikit-learn provides functions for splitting the data, encoding categorical values, and scaling numerical features.

2. Load the Iris Dataset

The Iris dataset was loaded into a Pandas DataFrame. After loading the dataset, its shape was checked to confirm the number of records and columns.

The dataset contains:

- 150 rows
- 6 columns

The columns include Id, four flower measurement features, and Species.

```
[1]:
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
[2]:
```

```
df = pd.read_csv("Iris.csv")

print("Dataset loaded successfully!")
print("Shape of dataset:", df.shape)
```

```
Dataset loaded successfully!
Shape of dataset: (150, 6)
```

3. Separate Features and Target

The dataset was divided into **features (X)** and a **target (y)**.

The following four measurements were selected as features:

- **SepalLengthCm**
- **SepalWidthCm**
- **PetalLengthCm**
- **PetalWidthCm**

The **Species** column was selected as the target variable.

The Id column was not included as a feature because it is only an identification number and does not represent a meaningful characteristic of the flower.

[3]:

```
features = [  
    "SepalLengthCm",  
    "SepalWidthCm",  
    "PetalLengthCm",  
    "PetalWidthCm"  
]  
  
X = df[features]  
y = df["Species"]  
  
print("Features (X):")  
print(X.head())  
  
print("\nTarget (y):")  
print(y.head())
```

Features (X):

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

Target (y):

0	Iris-setosa
1	Iris-setosa
2	Iris-setosa
3	Iris-setosa
4	Iris-setosa

Name: Species, dtype: object

4. Encode the Target Variable

The target variable, Species, contains categorical text values such as:

- Iris-setosa
- Iris-versicolor
- Iris-virginica

Since Machine Learning algorithms generally work with numerical values, the species names were converted into numerical labels using **LabelEncoder**.

For example, the three species are represented as different numerical classes.

This allows the Machine Learning model to process the target variable correctly.

[4]:

```
label_encoder = LabelEncoder()

y_encoded = label_encoder.fit_transform(y)

print("Original target values:")
print(y.head())

print("\nEncoded target values:")
print(y_encoded[:5])

print("\nClasses:")
print(label_encoder.classes_)
```

Original target values:

```
0    Iris-setosa
1    Iris-setosa
2    Iris-setosa
3    Iris-setosa
4    Iris-setosa
```

Name: Species, dtype: object

Encoded target values:

```
[0 0 0 0 0]
```

Classes:

```
['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
```

5. Split Data into Training and Testing Sets

The dataset was divided into **training data** and **testing data** using Scikit-learn's `train_test_split()` function.

An **80:20 split** was selected for this task.

This means:

- **80% → Training Data = 120 records**
- **20% → Testing Data = 30 records**

The `random_state` parameter was used so that the same split can be reproduced whenever the code is executed. The `stratify` parameter was also used to maintain a balanced representation of the three Iris species in both the training and testing datasets.

[5]:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)
```

```
print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
print("Training target shape:", y_train.shape)
print("Testing target shape:", y_test.shape)
```

```
Training data shape: (120, 4)
Testing data shape: (30, 4)
Training target shape: (120,)
Testing target shape: (30,)
```

6. Apply Feature Scaling

The numerical features were scaled using **StandardScaler** from Scikit-learn.

Feature scaling changes the numerical values to a comparable scale. This can be useful for Machine Learning algorithms that are sensitive to differences in feature magnitude.

The scaler was fitted only on the training data and then applied to both the training and testing data. This prevents information from the testing dataset from influencing the preprocessing of the training data.

```
[6]:
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Scaled training data:")
print(X_train_scaled[:5])

print("\nScaled testing data:")
print(X_test_scaled[:5])
```

```
Scaled training data:
[[-1.72156775 -0.32483982 -1.34703555 -1.32016847]
 [-1.12449223 -1.22612948  0.41429037  0.65186742]
 [ 1.14439475 -0.55016223  0.58474127  0.25746024]
 [-1.12449223  0.12580502 -1.29021859 -1.45163753]
 [-0.40800161 -1.22612948  0.13020555  0.12599118]]
```

```
Scaled testing data:
[[-1.72156775 -0.0995174  -1.40385252 -1.32016847]
 [ 0.30848902 -0.0995174  0.64155823  0.78333648]
 [-1.12449223 -1.45145189 -0.26751321 -0.268416  ]
 [-1.00507713 -1.67677431 -0.26751321 -0.268416  ]
 [-1.72156775  0.35112743 -1.40385252 -1.32016847]]
```

7. Final Dataset Preparation

After completing the train-test split and preprocessing, the final shapes of the datasets were checked.

The final prepared data contains:

Dataset	Number of Records	Features
Training Data	120	4
Testing Data	30	4

The target variables contain the corresponding species labels for the training and testing records.

[7]:

```
print("Final Dataset Preparation")
print("-----")
print("X_train:", X_train_scaled.shape)
print("X_test :", X_test_scaled.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
```

Final Dataset Preparation

X_train: (120, 4)

X_test : (30, 4)

y_train: (120,)

y_test : (30,)

8. Why Do We Divide Data into Training and Testing Sets?

Dividing the dataset into training and testing sets is an important step in Machine Learning.

Training Data → Model Learning

The training data is used to teach the Machine Learning model. The model learns patterns and relationships between the flower measurements and their species.

In this task, **120 Iris records** are used as training data.

Testing Data → Prediction

The testing data contains records that were not used during the training process. It is used to check how well the trained model can predict the species of previously unseen flowers.

In this task, **30 Iris records** are used as testing data.

Why is this important?

If the entire dataset is used for training, it becomes difficult to determine whether the model can perform well on new, unseen data.

By keeping some data separate for testing, we can evaluate the model's ability to **generalize** beyond the examples it learned from.

9. Training Data → Model Learning → Testing Data → Prediction

The complete process can be understood as:

Training Data → Model Learning → Testing Data → Prediction

First, the training data is given to the Machine Learning model so that it can learn patterns from the flower measurements.

After learning, the model receives the testing data, which it has not seen before. The model then uses the learned patterns to predict the species of the flowers.

This process helps us determine whether the Machine Learning model has learned useful patterns rather than simply memorizing the training data.

Conclusion

In this task, the Iris dataset was prepared for Machine Learning using Pandas and Scikit-learn. The dataset was divided into features and target variables, and the categorical target values were converted into numerical labels using LabelEncoder.

The dataset was then divided into **80% training data and 20% testing data**. Basic feature scaling was applied using StandardScaler to make the numerical features comparable.

As a result, the dataset is now properly prepared for the next stage of a Machine Learning project, where a model can be trained using the training data and evaluated using the testing data.

Final Workflow

Iris Dataset → Features & Target → Label Encoding → Train-Test Split → Feature Scaling → Model Training → Prediction